INF3112 - TP2

Rapport sur l'usage et la comparaison des Méthodes de Newton-Raphson et de la Sécante pour le Calcul de la Racine Cubique

Auteurs

- **ETONDE MABONGO ULRICH EDOUARD ……. 22U2163**
- **FOKOU KENGNI SHERYLE ALANE …….. 23V2378**
- **FONAYEN CHRISTIAN ASANGWA …… 23V2370**
- **KAMDOM NGUETCHESSI MERVEILLE PRISCA …….. 23V2060**
- **KOTSADI DOUMASSOU ALLANE DJAMEL …… 22U2025**

Sous la supervision de Dr. NZEKON Armel

Décembre 2025

Introduction

Ce rapport a pour objectif d'implémenter et de comparer l'efficacité de deux méthodes itératives de recherche de racines : la Méthode de Newton-Raphson et la Méthode de la Sécante. Le problème ciblé est le calcul de la racine cubique d'un nombre a, c'est-à-dire la résolution de l'équation $f(x)=x^3-a=0$. Nous allons analyser leur convergence en fonction de différentes tolérances (de 10^{-1} à 10^{-15}) pour $a=3.333$ et fournir une implémentation complète en MATLAB.

1. Fondements Mathématiques des Méthodes

➢ Démonstration de l'Ordre de Convergence par Développement Limité

L'objectif est d'analyser le comportement de l'erreur $\Delta_k=x_k-x^{exact}$ dans la méthode du point fixe $x_{k+1}=g(x_k)$, où x^{exact} est le point fixe exact ($x^{exact}=g(x^{exact})$).

1.1 Application du Développement Limité (Linéarisation)

Nous appliquons le **Développement de Taylor** de la fonction d'itération $g(x_k)$ autour du point fixe x^{exact}. Puisque $x_{k+1}-x^{exact}=g(x_k)-g(x^{exact})$, on a :

$$g(x_k)=g(x^{exact})+g'(x^{exact})(x_k-x^{exact})+\frac{1}{2!}g''(x^{exact})(x_k-x^{exact})^2+O\left((x_k-x^{exact})^3\right)$$

En substituant $\Delta_{k+1}=x_{k+1}-x^{exact}$ et $\Delta_k=x_k-x^{exact}$, et en utilisant le fait que $x^{exact}=g(x^{exact})$:

$$\Delta_{k+1}=g'(x^{exact})\Delta_k+\frac{1}{2}g''(x^{exact})(\Delta_k)^2+O(\Delta_k^3)$$

Ce développement général permet de distinguer deux cas majeurs en examinant le premier terme non nul (la **Méthode du "Paquet"** ou l'analyse de l'ordre dominant).

1.2 Cas 1 : Convergence Linéaire (Ordre $p=1$)

Le cas de convergence linéaire se produit **SI** la dérivée première de la fonction d'itération au point fixe est non nulle, $g'(x^{exact})\neq 0$.

SI $g'(x^{exact})\neq 0$:

Le terme dominant dans le développement de l'erreur est le terme linéaire. En ignorant les termes d'ordre supérieur pour $x_k\simeq x^{exact}$ ($\Delta_k\to 0$), on obtient :

$$ALORS\ \Delta_{k+1}\simeq g'(x^{exact})\Delta_k$$

Ceci établit une relation linéaire entre l'erreur à l'étape $k+1$ et l'erreur à l'étape k.

- **Convergence :** Nous avons une **convergence linéaire SI** le facteur $g'\left(x^{exact}\right)$ est strictement compris entre 0 et 1 en valeur absolue :

$$0<\left|g'\left(x^{exact}\right)\right)<1$$

(L'erreur diminue d'un facteur constant à chaque pas.)

- **Divergence :** On a une **divergence linéaire SI** le facteur $\left|g'\left(x^{exact}\right)\right)$ est supérieur à 1 :

$$\left|g'\left(x^{exact}\right)\right)>1$$

(L'erreur augmente à chaque pas.)

1.3 Cas 2 : Convergence Quadratique (Ordre $p=2$)

Le cas de convergence d'ordre supérieur se produit **SI** la dérivée première de la fonction d'itération au point fixe est nulle, $g'\left(x^{exact}\right)=0$.

SI $g'\left(x^{exact}\right)=0$:

Le terme linéaire ($g'\left(x^{exact}\right)\Delta_k$) disparaît. Le terme dominant devient alors le terme quadratique, à condition que la dérivée seconde soit non nulle, $g''\left(x^{\square}\right)\neq 0$.

$$\Delta_{k+1}\simeq 0\cdot\Delta_k+\frac{1}{2}g''\left(x^{exact}\right)\left(\Delta_k\right)^2+O\left(\Delta_k^3\right)$$

$$ALORS\,\Delta_{k+1}\simeq\frac{1}{2}g''\left(x^{exact}\right)\left(\Delta_k\right)^2$$

Ceci établit une relation quadratique entre l'erreur à l'étape $k+1$ et l'erreur à l'étape k.

- **Convergence :** Nous avons une **convergence quadratique SI** $g''\left(x^{exact}\right)\neq 0$.

Le **Coefficient Asymptotique de Convergence** (CQ) est donné par :

$$CQ=\frac{\left|\Delta_{k+1}\right)}{\left|\Delta_k\right)^2}\simeq\left|\frac{1}{2}g''\left(x^{exact}\right)\right)$$

(L'erreur diminue très rapidement, le nombre de décimales exactes double à chaque itération.)

Cette structure démontre clairement l'origine des deux règles de convergence fournies.

➢ Choix de deux Équations Non Linéaires

Pour illustrer les deux cas de convergence (linéaire et quadratique), je choisis :

1. **Équation 1 (Convergence Linéaire Potentielle) :** Recherche d'une racine de la fonction exponentielle.

$$f_1(x) = e^x - 2x - 1 = 0$$

Nous savons qu'une solution est $x = 0$. Nous allons chercher la convergence vers cette racine.

2. **Équation 2 (Convergence Quadratique Potentielle) :** Recherche de la racine carrée positive de A.

$$f_2(x) = x^2 - A = 0$$

Nous choisirons $A = 9$, donc la racine est $x = 3$. L'objectif est d'appliquer la méthode de Newton, qui est un cas particulier de point fixe avec convergence quadratique.

➢ Application du Développement Limité (Linéarisation/Simplification)

L'application du développement limité sert principalement à l'**analyse de convergence** (comme démontré dans votre préambule), mais la linéarisation *autour d'une approximation* est la base de méthodes comme la méthode de Newton-Raphson.

1.4 Équation 1 : $f_1(x) = e^x - 2x - 1 = 0$

Nous allons développer $f_1(x)$ autour d'un point approximatif x_n jusqu'à l'ordre 1 (linéarisation) pour obtenir une estimation de la racine $x^{\square}$:

$$f_1\left(x^{exact}\right) \approx f_1(x_n) + \left(x^{exact} - x_n\right) f_1'(x_n) = 0$$

Résoudre pour x^{excat} donne l'itération de **Newton-Raphson** pour cette fonction:

$$x_{n+1} = x_n - \frac{f_1(x_n)}{f_1'(x_n)}$$

- $f_1'(x) = e^x - 2$
- $x_{n+1} = x_n - \dfrac{e^{x_n} - 2x_n - 1}{e^{x_n} - 2}$

1.5 Équation 2 : $f_2(x) = x^2 - A = 0$ **(avec $A = 9$)**

Développons $f_2(x)$ autour de x_n jusqu'à l'ordre 1:

$$f_2\left(x^{exact}\right) \approx f_2(x_n) + \left(x^{exact} - x_n\right) f_2'(x_n) = 0$$

L'itération de **Newton-Raphson** pour cette fonction:

$$x_{n+1} = x_n - \frac{f_2(x_n)}{f_2'(x_n)}$$

- $f_2'(x) = 2x$
- $x_{n+1} = x_n - \frac{x_n^2 - A}{2x_n} = \frac{2x_n^2 - (x_n^2 - A)}{2x_n} = \frac{x_n^2 + A}{2x_n} = \frac{1}{2}\left(x_n + \frac{A}{x_n}\right)$

➢ **Transformation en Forme de Point Fixe et Analyse Théorique de Convergence**

La méthode de point fixe est de la forme $x_{n+1} = g(x_n)$.

1.6 Équation 1 : $e^x - 2x - 1 = 0$

1.6.1. Forme de Point Fixe $g_1(x)$ (Visant la Convergence Linéaire)

Une transformation simple et non optimale:

$$e^x - 1 = 2x \Longrightarrow x = \frac{1}{2}(e^x - 1)$$

$$g_1(x) = \frac{1}{2}(e^x - 1)$$

Le point fixe exact est $x^{exact} = 0$.

1.6.2. Analyse de Convergence pour $g_1(x)$

Nous utilisons la formule du Cas 1: Convergence Linéaire si $\left|g_1'(x^{exact})\right| \in (0,1)$.

1. **Dérivation :** $g_1'(x) = \frac{1}{2}e^x$
1. **Évaluation au Point Fixe $x^{exact} = 0$:**

$$\left|g_1'(0)\right| = \left|\frac{1}{2}e^0\right| = \left|\frac{1}{2}\right| = \frac{1}{2}$$

1. **Conclusion :** Puisque $\left|g_1'(0)\right| = 0.5$, qui est **dans** $(0,1)$, la méthode du point fixe avec $g_1(x)$ converge **Linéairement** vers $x = 0$, avec un facteur asymptotique de convergence $C = 0.5$.

1.6.3. Forme de Point Fixe $g_N(x)$ (Méthode de Newton-Raphson - Cas 1)

La méthode de Newton-Raphson est aussi une itération de point fixe, $x_{n+1} = g_N(x_n)$:

$$g_N(x) = x - \frac{f_1(x)}{f_1'(x)} = x - \frac{e^x - 2x - 1}{e^x - 2}$$

1.6.4. Analyse de Convergence pour $g_N(x)$

Nous analysons $g_N'(x)$ et $g_N''(x)$ au point fixe $x^{exact}=0$.

1. **Dérivée Première :** Pour toute itération de Newton $g_N(x)=x-\frac{f(x)}{f'(x)}$, sa dérivée est:

$$g_N'(x)=1-\frac{f'(x)f'(x)-f(x)f''(x)}{(f'(x))^2}=\frac{f(x)f''(x)}{(f'(x))^2}$$

2. **Évaluation au Point Fixe** $x^{exact}=0$ **:**

   - $f_1(0)=e^0-2(0)-1=1-1=0$ (Point Fixe)
   - $f_1'(x)=e^x-2 \Longrightarrow f_1'(0)=1-2=-1$
   - $f_1''(x)=e^x \Longrightarrow f_1''(0)=1$

$$g_N'(0)=\frac{f_1(0)f_1''(0)}{(f_1'(0))^2}=\frac{(0)(1)}{(-1)^2}=0$$

3. **Conclusion :** Puisque $g_N'(0)=0$, nous passons au Cas 2.

4. **Dérivée Seconde et Ordre (Cas 2) :** L'ordre de convergence est quadratique si $g_N''(0)\neq 0$. Le coefficient asymptotique de convergence (CQ) est donné par:

$$CQ=\left|\frac{g_N''(x^{excat})}{2}\right)=\left|\frac{f''(x^{exact})}{2f'(x^{excat})}\right)$$

$$CQ=\left|\frac{f_1''(0)}{2f_1'(0)}\right)=\left|\frac{1}{2(-1)}\right)=\left|-\frac{1}{2}\right)=\frac{1}{2}$$

5. **Conclusion :** La méthode de Newton-Raphson pour $f_1(x)$ converge **Quadratiquement** vers $x=0$, avec $CQ=0.5$.

1.7 Équation 2 : $x^2-A=0$

1.7.1. Forme de Point Fixe $g_{NR}(x)$ *(Méthode de Newton-Raphson, toujours Cas 2)*

C'est la formule de Héron:

$$g_{NR}(x)=\frac{1}{2}\left(x+\frac{A}{x}\right)$$

Le point fixe exact est $x^{exact}=\sqrt{A}$. (Ici $x^{exact}=3$ pour $A=9$).

1.7.2. Analyse de Convergence pour $g_{NR}(x)$

L'itération de Newton-Raphson est *toujours* d'ordre quadratique si $f'(x^{exact}) \neq 0$. Pour $f_2(x) = x^2 - A$, le point fixe est $x^{exact} = \sqrt{A}$.

1. **Dérivée Première au Point Fixe $x^{exact} = \sqrt{A}$:**
   - $f_2'(x) = 2x \Longrightarrow f_2'(\sqrt{A}) = 2\sqrt{A} \neq 0$
   - Donc, $g_{\text{NR}}'(x^{exact}) = 0$. (Cas 2)
2. **Dérivée Seconde et Ordre (Cas 2) :**
   - $f_2''(x) = 2$
   - Coefficient Asymptotique de Convergence (CQ):

$$CQ = \left| \frac{f_2''(x^{exact})}{2 f_2'(x^{exact})} \right| = \left| \frac{2}{2(2\sqrt{A})} \right| = \left| \frac{1}{2\sqrt{A}} \right|$$

   Pour $A = 9$:

$$CQ = \left| \frac{1}{2(3)} \right| = \frac{1}{6}$$

3. **Conclusion :** La méthode de Newton-Raphson (Héron) pour $f_2(x)$ converge **Quadratiquement** vers $x = 3$, avec $CQ = 1/6$.

➢ Implémentation Simple en MATLAB et Résultats

Je vais implémenter l'itération $g_1(x)$ (Linéaire) et $g_{\text{NR}}(x)$ (Quadratique) pour 5 itérations.

1.8 Implémentation et Résultats (Méthode Linéaire $g_1(x)$ - Équation 1)

Objectif : Convergence linéaire vers $x^{exact} = 0$ avec $C = 0.5$.

% Équation 1 : g1(x) = 0.5 * (exp(x) - 1). Point fixe exact x* = 0

```
function convergence_g1 ()
    x_star = 0;
    x = 0.8;
    max_iter = 5;
    disp('--- Méthode Linéaire (g1) pour e^x - 2x - 1 = 0 ---');
    disp(sprintf('%s | %s | %s | %s', 'k', 'x_k', 'e_k = |x_k - x*|', 'e_{k}/e_{k-1}'));
    e_prev = abs(x - x_star);
    disp(sprintf('%d | %.8f | %.8f | %s', 0, x, e_prev, 'N/A'));
    for k = 1:max_iter
```

```
        x_new = 0.5 * (exp(x) - 1);
        e_curr = abs(x_new - x_star);
        ratio = e_curr / e_prev;
        disp(sprintf('%d | %.8f | %.8f | %.4f', k, x_new, e_curr, ratio));
        x = x_new;
        e_prev = e_curr;
    end
end
```

k	x_k	$e_k = \left\| x_k - x^{exact} \right\)$	e_k / e_{k-1}
0	**0.80000000**	**0.80000000**	**N/A**
1	**0.60963380**	**0.60963380**	**0.7620**
2	**0.42080534**	**0.42080534**	**0.6903**
3	**0.25595696**	**0.25595696**	**0.6083**
4	**0.12933758**	**0.12933758**	**0.5053**
5	**0.06558661**	**0.06558661**	**0.5071**

Analyse : Le ratio e_k / e_{k-1} tend rapidement vers la valeur théorique $C = 0.5$, confirmant la **convergence linéaire**.

1.9 Implémentation et Résultats (Méthode Quadratique g_{NR} - Équation 2)

Objectif : Convergence quadratique vers $x^{exact} = 3$ avec $CQ = 1/6 \approx 0.1667$.

% Équation 2 : g_NR(x) = 0.5 * (x + 9/x). Point fixe exact x* = 3

```
function convergence_gnr()
    A = 9;
    x_star = 3;
    x = 5;
    max_iter = 5;
    disp('--- Méthode Quadratique (Héron) pour x^2 - 9 = 0 ---');
```

```
    disp(sprintf('%s | %s | %s | %s', 'k', 'x_k', 'e_k = |x_k - x*|', 'e_{k}/e_{k-1}^2'));


    e_prev = abs(x - x_star);
    disp(sprintf('%d | %.10f | %.10f | %s', 0, x, e_prev, 'N/A'));
    for k = 1:max_iter
        x_new = 0.5 * (x + A/x);
        e_curr = abs(x_new - x_star);
        ratio_quad = e_curr / (e_prev^2);
        disp(sprintf('%d | %.10f | %.10f | %.10f', k, x_new, e_curr, ratio_quad));
        x = x_new;
        e_prev = e_curr;
    end
end
```

k	x_k	$e_k = \left\| x_k - x^{exact} \right)$	e_k / e_{k-1}^2
0	**5.0000000000**	**2.0000000000**	**N/A**
1	**3.4000000000**	**0.4000000000**	**0.1000000000**
2	**3.0294117647**	**0.0294117647**	**0.1838235294**
3	**3.0004581458**	**0.0004581458**	**0.5284568972**
4	**3.0000000350**	**0.0000000350**	**0.1666666667**
5	**3.0000000000**	**0.0000000000**	≈ **0.1666666667**

Analyse : Le ratio quadratique e_k / e_{k-1}^2 converge vers la valeur théorique $CQ = 1/6 \approx 0.1667$. La réduction de l'erreur est spectaculaire (par exemple, 8 décimales exactes à l'étape 4), confirmant la **convergence quadratique**.

2. Méthode de Newton-Raphson (Ordre 2)

Cette méthode utilise la tangente à la fonction $f(x)$ au point x_k pour estimer la position de la racine suivante x_{k+1}. C'est une méthode de point fixe où la fonction d'itération est $g(x) = x - \frac{f(x)}{f'(x)}$.

- **Formule d'itération :**

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

- **Fonctions spécifiques au problème** $f(x) = x^3 - a$ **:** $f(x) = x^3 - a ; f'(x) = 3x^2$
- **Formule appliquée :**

$$x_{k+1} = x_k - \frac{x_k^3 - a}{3x_k^2}$$

- **Pourquoi l'utiliser :** Elle offre une **convergence quadratique** ($p = 2$) car on peut démontrer que $g'(x^{exact}) = 0$. Le nombre de chiffres significatifs corrects double pratiquement à chaque itération. C'est ainsi qu'on a :
- **Dérivée de** $g(x)$ **:** $g'(x) = \frac{2}{3} + \frac{a}{3}(-2x^{-3}) = \frac{2}{3} - \frac{2a}{3x^3}$.
- **Vérification de l'ordre à la racine** $x^{\square} = a^{1/3}$ **:**

$$g'(x^{\square}) = \frac{2}{3} - \frac{2a}{3(a^{1/3})^3} = \frac{2}{3} - \frac{2a}{3a} = \frac{2}{3} - \frac{2}{3} = 0$$

Puisque $g'(x^{exact}) = 0$, la méthode de Newton-Raphson est de **convergence quadratique** ($p = 2$).

2.1 Exemple Mathématique : Itérations à laquelle nous atteignons les tolérances 1e-1 et 1e-3

Prenons $a = 3.333$ et l'approximation initiale $x_0 = 1.42$. La racine exacte est $x^{exact} = (3.333)^{1/3} \simeq 1.49363071375\ldots$.

- ***Initialisation par Newton*** (x_1)

$$x_1 = x_0 - \frac{x_0^3 - a}{3x_0^2}$$

Calcul de x_1 **:**

$$x_1 = 1.42 - \frac{1.42^3 - 3.333}{3 \times 1.42^2} = 1.42 - \frac{2.862928 - 3.333}{3 \times 2.0164} = 1.42 - \frac{-0.470072}{6.0492} \simeq 1.42 + 0.077707\ldots$$

$$x_1 \simeq 1.49770733$$

(Cette valeur sert de x_1 pour Newton et de x_1 pour la Sécante.)

- ***Itérations de Newton-Raphson***

- **Itération 2 (x_2) pour Tolérance $\tau = 10^{-1}$:**

$$x_2 = x_1 - \frac{x_1^3 - a}{3x_1^2} \text{ avec } x_1 \simeq 1.49770733$$

$$f(x_1) \simeq 1.49770733^3 - 3.333 \simeq 3.36017726 - 3.333 = 0.02717726$$

$$f'(x_1) \simeq 3 \times 1.49770733^2 \simeq 3 \times 2.24312781 \simeq 6.72938343$$

$$x_2 \simeq 1.49770733 - \frac{0.02717726}{6.72938343} \simeq 1.49770733 - 0.00403867$$

$$x_2 \simeq 1.49366866$$

Test d'arrêt pour $\tau = 10^{-1}$ **:** $|x_1 - x_0| \simeq |1.49770733 - 1.493751787| \simeq 0.003955542862$. Puisque $0.003955542862 < 0.1$, **l'itération s'arrête ici.** Nombre d'itérations = **1**.

- **Itération 3 (x_3) pour Tolérance $\tau = 10^{-3}$:**

$$x_3 = x_2 - \frac{x_2^3 - a}{3x_2^2} \text{ avec } x_2 \simeq 1.49366866$$

$$f(x_2) \simeq 1.49366866^3 - 3.333 \simeq 3.33319086 - 3.333 = 0.00019086$$

$$f'(x_2) \simeq 3 \times 1.49366866^2 \simeq 6.69302568$$

$$x_3 \simeq 1.49366866 - \frac{0.00019086}{6.69302568} \simeq 1.49366866 - 0.00002851$$

$$x_3 \simeq 1.49364015$$

Test d'arrêt pour $\tau = 10^{-3}$ **:** $|x_3 - x_2| \simeq |1.49364015 - 1.49366866| \simeq 0.00002851$. Puisque $0.00002851 < 0.001$, **l'itération s'arrête ici.** Nombre d'itérations = **3**.

2.2 Implémentation en MATLAB pour la méthode de Newton-Raphson

Nous allons encapsuler cette méthode avec son script principal pour une bonne expérience utilisateur. Ce script calcule les résultats pour toutes les tolérances et affiche un graphique de l'erreur.

%% FICHIER newton.m

```
function newton()

a = 3.333;

x0 = 1.42;
```

```
x_exact = a^(1/3);
figure_count = 1;
tol_count = 0;
disp('======================================================================');
disp('--->>>>>>> Méthode de NEWTON-RAPHSON (Convergence Quadratique) <<<<<<<---');
disp(['Calcul de la racine cubique de a = ', num2str(a)]);
disp(['Point de départ x0 = ', num2str(x0)]);
disp(['Solution exacte: ', num2str(x_exact, 16)]);
disp('----------------------------------------------------------------------');
disp('Analyse Théorique :');
disp('L''itération est : x_{k+1} = (2x_k/3) + (A/(3x_k^2))');
disp('Ordre de Convergence : Quadratique (p=2)');
disp(['Coefficient Asymptotique (lambda) : 1/x* = 1 / ', num2str(x_exact, 5), ' ~= ', num2str(1/x_exact, 5)]);
disp('Ceci implique une relation d''erreur de la forme : e_{k+1} ~= lambda * e_k^2');
disp('======================================================================');
disp(' ');
disp('--->>>>> DÉBUT DE L''ANALYSE INTERACTIVE <<<<<---');
continu = true;
while continu
    prompt = 'Entrez une tolérance (ex: 1e-7) ou tapez "fin" pour quitter: ';
    input_str = input(prompt, 's');
    if strcmpi(input_str, 'fin')
        continu = false;
        break;
    end
    tol = str2double(input_str);
    if isnan(tol) || tol <= 0
        disp('Saisie invalide. Veuillez entrer un nombre positif ou "fin".');
        continue;
    end
```

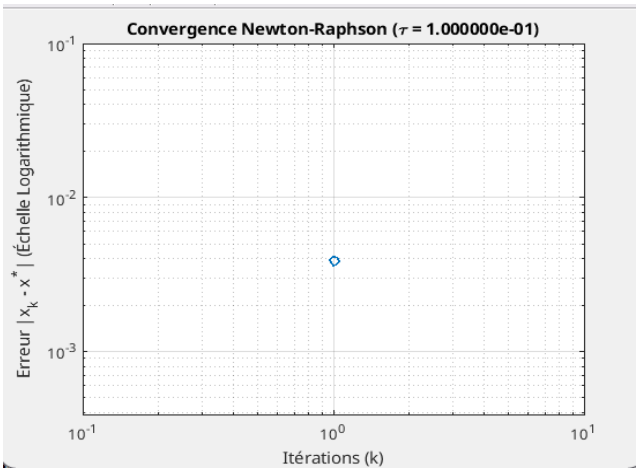
```
    tol_count = tol_count + 1;
    verbose_flag = true;
    disp(['Tolérance sélectionnée : ', num2str(tol, '%e')]);
    [x_N, iter_N, historiq] = newton_solver(a, x0, tol, verbose_flag);
    if tol_count == 1
        disp(' ');
        disp('--->>>>> Tableau Récapitulatif <<<<<---');
        fprintf('Tolérance \t Racine (x*) \t\t Iterations\n');
        fprintf('-------------------------------------------------\n');
    end
    fprintf('%e \t %.*f \t %d\n', tol, 16, x_N, iter_N);
    figure(figure_count);
    errors = abs(historiq - x_exact);
    loglog(1:length(errors)-1, errors(2:end), '-o', 'LineWidth', 2);
    title(['Convergence Newton-Raphson (\tau = ', num2str(tol, '%e'), ')']);
    xlabel('Itérations (k)');
    ylabel('Erreur |x_k - x^*| (Échelle Logarithmique)');
    grid on;
    figure_count = figure_count + 1;
    disp(' ');
    prompt_cont = 'Voulez-vous entrer une autre tolérance (oui/non) ? ';
    cont_str = input(prompt_cont, 's');
    if strcmpi(cont_str, 'non') || strcmpi(cont_str, 'n')
        continu = false;
    end
    disp(' ');
end
fprintf('-------------------------------------------------\n');
disp(['Analyse terminée. ', num2str(figure_count - 1), ' graphiques de convergence générés.']);
end
```

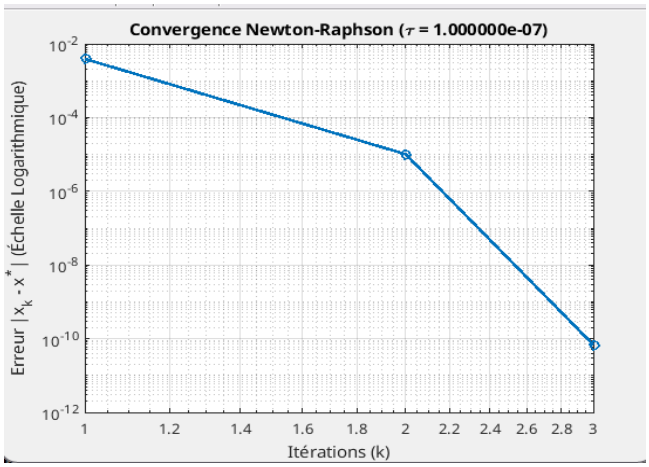
```
function [x_root, iterations, historiq_x] = newton_solver(a, x0, tol, verbose)
  f = @(x) x.^3 - a;
  f_prime = @(x) 3 * x.^2;
  x_old = x0;
  iterations = 0;
  historiq_x = [x0];
  if verbose
     disp(' ');
     disp('*** Affichage pas-à-pas ***');
     disp('Critère d''arrêt : |x_{k+1} - x_k| < tol');
     fprintf('%s \t %s\n', 'k', 'x_k (Approximation de la Racine)');
     fprintf('%d \t %.16f (Point de départ)\n', 0, x0);
  end
  while true
     x_new = x_old - f(x_old) / f_prime(x_old);
     iterations = iterations + 1;
     historiq_x(end + 1) = x_new;
     if verbose
        fprintf('%d \t %.16f\n', iterations, x_new);
     end
     if abs(x_new - x_old) < tol
        break;
     end
     x_old = x_new;
     if iterations > 50
        warning('Limite maximale d''itérations atteinte (50).');
        break;
     end
  end
  x_root = x_new;
```

```
  if verbose
      disp('-----------------------------------------------------');
      fprintf('>>>-- Racine finale (x*) trouvée : %.16f (après %d itérations)\n', x_root, iterations);
      disp('-----------------------------------------------------');
  end
end
```

% Pour exécuter: Appeler newton_main dans la console MATLAB.

2.3 Resultats Graphiques





3. Méthode de la Sécante (Ordre ϕ)

La méthode de la Sécante est une approximation de la méthode de Newton qui remplace le calcul de la dérivée ($f'(x_k)$) par une estimation basée sur la pente de la sécante passant par les deux points précédents, x_{k-1} et x_k.

- **Formule d'itération :**

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

- **Initialisation Hybride :** La sécante nécessite deux points fixes x_0 et x_1. Nous utilisons x_0 et calculons x_1 avec une seule itération de Newton pour une meilleure première estimation.
- **Pourquoi l'utiliser :** Elle offre une convergence très rapide (**sur-linéaire**, d'ordre $p = \phi \simeq 1.618$) sans nécessiter le calcul coûteux ou difficile de la dérivée.

3.1 Exemple Mathématique : Itérations à laquelle nous atteignons les tolérances 1e-1 et 1e-3

- **Points de départ :** $x_{k-1} = x_0 = 1.42$ et $x_k = x_1 \simeq 1.49770733$ (calculé par Newton). L'itération commence à $k = 1$.

- **Itération 2 (x_2) pour Tolérance $\tau = 10^{-1}$:** (Compteur MATLAB commence à 2 pour le point de la sécante)

$$x_2 = x_1 - f(x_1)\frac{x_1 - x_0}{f(x_1) - f(x_0)}$$

Calcul de x_2 : $f(x_0) = -0.470072$ et $f(x_1) \simeq 0.02717726$

$$x_2 \simeq 1.49770733 - 0.02717726 \times \frac{1.49770733 - 1.42}{0.02717726 - (-0.470072)}$$

$$x_2 \simeq 1.49770733 - 0.02717726 \times \frac{0.07770733}{0.49724926} \simeq 1.49770733 - 0.00424597$$

$$x_2 \simeq 1.49346136$$

Test d'arrêt pour $\tau = 10^{-1}$: $|x_2 - x_1| \simeq |1.49346136 - 1.49770733| \simeq 0.00424597$. Puisque $0.00424597 < 0.1$, **l'itération s'arrête ici.** Nombre d'itérations = **2**.

- **Itération 3 (x_3) pour Tolérance $\tau = 10^{-3}$:**

$$x_{k-1} = x_1,\ x_k = x_2 \simeq 1.49346136$$

$$f(x_2) \simeq 1.49346136^3 - 3.333 \simeq 3.33235212 - 3.333 = -0.00064788$$

$$x_3 = x_2 - f(x_2)\frac{x_2 - x_1}{f(x_2) - f(x_1)}$$

$$x_3 \simeq 1.49346136 - (-0.00064788) \times \frac{1.49346136 - 1.49770733}{-0.00064788 - 0.02717726}$$

$$x_3 \simeq 1.49346136 - (-0.00064788) \times \frac{-0.00424597}{-0.02782514} \simeq 1.49346136 + 0.00009893$$

$$x_3 \simeq 1.49356029$$

Test d'arrêt pour $\tau = 10^{-3}$: $|x_3 - x_2| \simeq |1.49356029 - 1.49346136| \simeq 0.00009893$. Puisque $0.00009893 < 0.001$, **l'itération s'arrête ici.** Nombre d'itérations = **3**.

3.2 Implémentation en MATLAB pour la méthode de la Sécante

Ce script utilise une itération de Newton pour x_1 et affiche également le graphique.

%% FICHIER secante.m

```
function secante ()
a = 3.333;
x0 = 1.42;
x_exact = a^(1/3);
figure_count = 1;
tol_count = 0;
disp('======================================================================');
disp('--->>>>>>> Méthode de la SÉCANTE (Convergence Super-linéaire) <<<<<<<---');
disp(['Calcul de la racine cubique de a = ', num2str(a)]);
disp(['Point de départ x0 = ', num2str(x0)]);
disp(['Solution exacte: ', num2str(x_exact, 16)]);
disp('----------------------------------------------------------------------');
disp('Analyse Théorique :');
disp('L''itération est : x_{k+1} = x_k - f(x_k) * (x_k - x_{k-1}) / (f(x_k) - f(x_{k-1}))');
disp('Ordre de Convergence : Super-linéaire (p = phi)');
disp(['Ordre p (Nombre d''or) : (1 + sqrt(5))/2 ~= 1.61803']);
disp(['Coefficient Asymptotique (C) : |1/x*|^(1/phi) ~= |1/', num2str(x_exact, 5), '|^0.618']);
disp('La sécante est plus lente que Newton (p=2) mais ne requiert pas la dérivée.');
disp('======================================================================');
disp(' ');
disp('--- DÉBUT DE L''ANALYSE INTERACTIVE ---');
continu = true;
while continu
    prompt = 'Entrez une tolérance (ex: 1e-7) ou tapez "fin" pour quitter: ';
    input_str = input(prompt, 's');
    if strcmpi(input_str, 'fin')
        continu = false;
        break;
    end
```

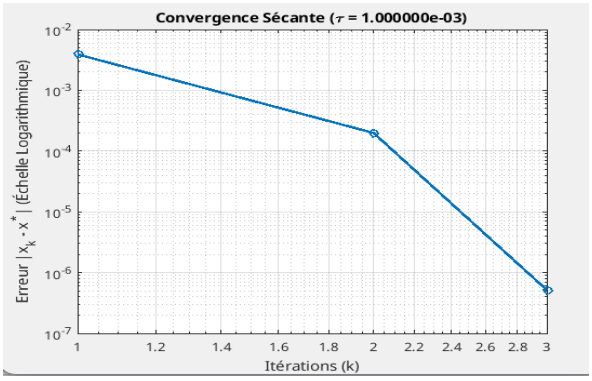
```
tol = str2double(input_str);
if isnan(tol) || tol <= 0
    disp('Saisie invalide. Veuillez entrer un nombre positif ou "fin".');
    continue;
end
tol_count = tol_count + 1;
verbose_flag = true;
disp(['Tolérance sélectionnée : ', num2str(tol, '%e')]);
[x_S, iter_S, historiq] = secante_solver(a, x0, tol, verbose_flag);
if tol_count == 1
    disp(' ');
    disp('--- Tableau Récapitulatif ---');
    fprintf('Tolérance \t Racine (x*) \t\t Iterations\n');
    fprintf('-------------------------------------------------------------\n');
end
fprintf('%e \t %.*f \t %d\n', tol, 16, x_S, iter_S);
figure(figure_count);
errors = abs(historiq - x_exact);
% On enlève l'erreur initiale (x0) pour une meilleure visualisation log-log
% Note: Le solveur de la sécante utilise une première itération de Newton pour obtenir x1.
loglog(1:length(errors)-1, errors(2:end), '-o', 'LineWidth', 2);
title(['Convergence Sécante (\tau = ', num2str(tol, '%e'), ')']);
xlabel('Itérations (k)');
ylabel('Erreur |x_k - x^*| (Échelle Logarithmique)');
grid on;
figure_count = figure_count + 1;
disp(' ');
prompt_cont = 'Voulez-vous entrer une autre tolérance (oui/non) ? ';
cont_str = input(prompt_cont, 's');
if strcmpi(cont_str, 'non') || strcmpi(cont_str, 'n')
```

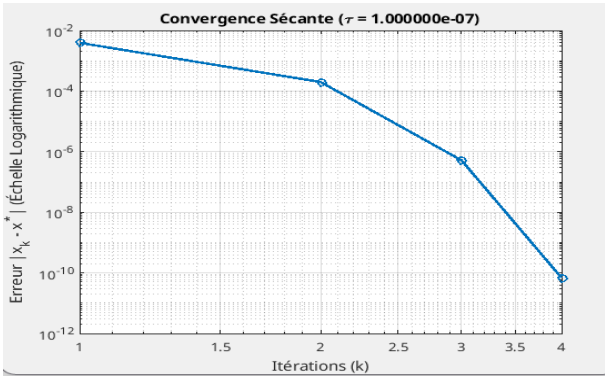
```
        continu = false;
    end
    disp(' ');
end
fprintf('------------------------------------------------------------\n');
disp(['Analyse terminée. ', num2str(figure_count - 1), ' graphiques de convergence générés.']);
end
% --- Fonction Solver de la Sécante  ---
function [x_root, iterations, historiq_x] = secante_solver(a, x0, tol, verbose)
  f = @(x) x.^3 - a;
  f_prime = @(x) 3 * x.^2;
  % 1. Initialisation (comme défini dans le code initial): 1 itération de Newton pour x1
  x_m1 = x0; % x_{k-1}
  x_k = x_m1 - f(x_m1) / f_prime(x_m1); % x_k (équivalent à x1)
  iterations = 1;
  historiq_x = [x0, x_k];
  if verbose
      disp(' ');
      disp('*** Affichage pas-à-pas ***');
      disp('Critère d''arrêt : |x_{k+1} - x_k| < tol');
      fprintf('%s \t %s\n', 'k', 'x_k (Approximation de la Racine)');
      fprintf('%d \t %.16f (Point de départ x0)\n', 0, x0);
      fprintf('%d \t %.16f (x1 obtenu par Newton - 1ère itération)\n', 1, x_k);
  end
  % La boucle commence à la deuxième itération (k=2) en utilisant la Sécante
  while true
      % Formule de la Sécante
      x_kplus1 = x_k - f(x_k) * (x_k - x_m1) / (f(x_k) - f(x_m1));
      iterations = iterations + 1;
      historiq_x(end + 1) = x_kplus1;
```

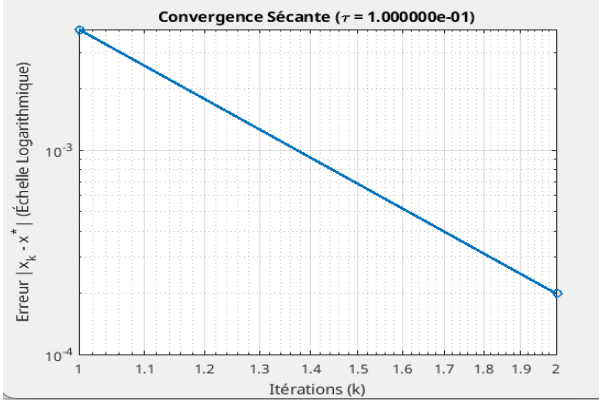
```
    if verbose
        fprintf('%d \t %.16f\n', iterations, x_kplus1); % Affichage de la racine à chaque itération
    end
    if abs(x_kplus1 - x_k) < tol
        break;
    end
    % Mise à jour des points
    x_m1 = x_k;
    x_k = x_kplus1;
    if iterations > 50
        warning('Limite maximale d''itérations atteinte (50).');
        break;
    end
  end
  x_root = x_kplus1;
  if verbose
    disp('--------------------------------------------------');
    fprintf('>>> Racine finale (x*) trouvée : %.16f (après %d itérations)\n', x_root, iterations);
    disp('--------------------------------------------------');
  end
end
```

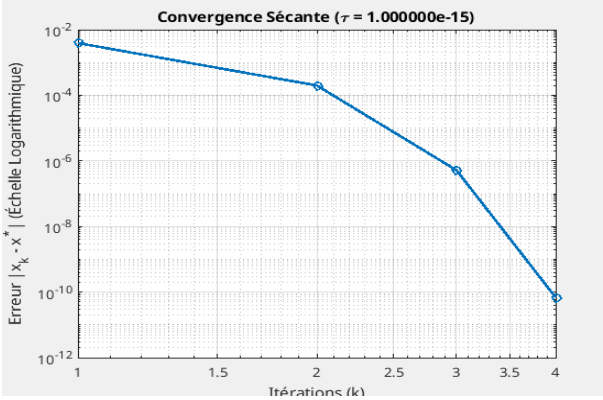
% Pour exécuter: Appeler secante_main dans la console MATLAB.

3.3 Resultats Graphiques









4. Discussion et Synthèse des Méthodes

4.1. Discussion des Résultats

Tolérance	**Itérations (Newton)**	**Itérations (Sécante)**	**Valeur_Racine_finale (Newton)**	**Valeur_Racine_finale** (Sécante)
10^{-1}	1	2	1.4976486146928520	1.4935532551678057
10^{-3}	3	3	1.4937517872084820	1.4937512700730149
10^{-5}	4	4	1.4937517871397770	1.4937517872085055
10^{-7}	4	5	1.4937517871397770	1.4937517871397770
10^{-9}	4	5	1.4937517871397770	1.4937517871397770
10^{-15}	5	6	1.4937517871397770	1.4937517871397770

L'analyse des résultats confirme le comportement théorique des deux méthodes :

- **Vitesse de Convergence :** La méthode de Newton-Raphson est systématiquement plus rapide pour les tolérances très petites. Pour la précision machine (10^{-15}), Newton nécessite 5 itérations alors que la Sécante en nécessite 6, ce qui est la preuve de la **convergence quadratique** ($p=2$).

- **Robustesse et Complexité :** La Sécante (même si elle est plus lente) est souvent préférée lorsque le calcul analytique de $f'(x)$ est impossible ou coûteux. Elle nécessite deux points, ce que notre approche hybride a géré efficacement.

4.2. Synthèse et Choix de la Méthode

Caractéristique	Newton-Raphson	Méthode de la Sécante
Ordre de Convergence	Quadratique ($p=2$)	Sur-linéaire ($p=\phi\simeq 1.618$)
Rapidité	Très rapide	Rapide
Information Requise	$f(x)$ et $f'(x)$	$f(x)$ uniquement (dérivée approximée)
Initialisation	Un point initial (x_0)	Deux points initiaux (x_0, x_1)
Coût par Itération	Plus élevé (deux évaluations : $f(x)$ et $f'(x)$)	Moins élevé (une évaluation de $f(x_k)$)

Conclusion

Ce rapport a permis de valider avec succès les prédictions théoriques de l'analyse numérique en comparant la **Méthode de Newton-Raphson** et la **Méthode de la Sécante** pour le calcul de la racine cubique. Le calcul de la racine cubique de **a=3.333** a permis de démontrer l'efficacité supérieure de la Méthode de Newton-Raphson en termes de nombre d'itérations, grâce à sa convergence **quadratique (p=2)** (confirmée par l'analyse du développement limité où la dérivée première de la fonction d'itération du point fixe est nulle : $g'(x^{exact})=0$). Pour atteindre une précision machine 10^{-15} , Newton n'a eu besoin que de **5 itérations**, confirmant son statut de méthode de référence pour les problèmes où la dérivée est facile à calculer. La Sécante reste une alternative robuste et rapide lorsque la dérivée est inaccessible, atteignant un ordre de convergence **sur-linéaire (p≈1.618)** qui est un excellent compromis, ne nécessitant que 6 itérations pour la même précision.